

DocuVector Lite

Plano de Validação Pré-Frontend

Sprint 4 completa · Antes de iniciar Sprint 5 (Frontend Jinja2+HTMX)

Objetivo deste documento

- Garantir que o sistema funciona como produto antes de construir o frontend.
- Cobrir os 3 modos de LLM: Mock (CI/offline), Ollama (local) e OpenAI (cloud).
- Validar o pipeline completo: upload → ingestão → RAG → benchmark → dashboard → audit.
- Documentar resultado esperado de cada etapa para uso como roteiro na defesa.

Item	Detalhe
Sprints concluídas	1, 2, 3 (RAG+LLM pluggable), 4A (compressão), 4B (dashboard)
Próxima sprint	5 — Frontend Jinja2+HTMX jurídico
Progresso estimado	~83% do projeto total
Pré-requisito mínimo	just ci verde + banco rodando + seed-admin

1. Pré-requisitos e Configuração do Ambiente

1.1 Dependências do sistema

1	Banco Postgres just up && just migrate-status	head = 0006_compression_nullable
2	Seed do admin just seed-admin	admin criado (idempotente)
3	Dependências just sync	Resolved N packages, sem erro
4	Gates CI just ci	CI local OK

1.2 Configuração do .env por modo de LLM

O sistema suporta 3 modos de geração de resposta, alternados via **LLM_DEFAULT_PROVIDER** no **.env** sem necessidade de alterar código. Cada modo tem comportamento distinto descrito nas seções seguintes:

Valor	Modo	Comportamento	Configuração necessária
mock	CI/dev offline	Resposta determinística. Sem rede. Sem custo. Tokens=0.	LLM_DEFAULT_PROVIDER=mock
ollama	Local	Modelo local qwen2.5:7b. Sem custo. Sem envio de dados.	LLM_DEFAULT_PROVIDER=ollama + ollama serve
openai	Cloud/premium	gpt-4o-mini via API. Custo por token. Latência ~500-1500ms.	LLM_DEFAULT_PROVIDER=openai + OPENAI_API_KEY

1.3 Instalação do Ollama (modo local)

```
# Linux / WSL
sudo apt-get install zstd
curl -fsSL https://ollama.com/install.sh | sh

# Baixar modelo padrão (4.7GB)
ollama pull qwen2.5:7b

# Modelo alternativo para máquina com < 8GB RAM
ollama pull llama3.2:3b
# Se usar llama3.2:3b, atualizar no .env:
# OLLAMA_MODEL=llama3.2:3b
```

1.4 Iniciar o servidor

```
# Terminal 1 – servidor FastAPI
just dev
# Esperado: Uvicorn running on http://127.0.0.1:8000

# Terminal 2 – Ollama (somente para modo local)
ollama serve
# Esperado: Listening on 127.0.0.1:11434

# Swagger UI
```

Abrir no browser: <http://127.0.0.1:8000/docs>

2. Como Usar o Swagger UI

O Swagger UI em **http://127.0.0.1:8000/docs** é a interface principal de validação manual. Todos os endpoints exigem autenticação JWT. O fluxo de autorização é:

1	Abrir Swagger http://127.0.0.1:8000/docs no browser	Página com todas as rotas agrupadas por tag
2	Login POST /api/v1/auth/login → preencher email e senha do admin	200 OK com access_token (JWT)
3	Autorizar Clicar no cadeado 'Authorize' no topo → colar o token	Cadeado fecha; requisições subsequentes incluem Bearer
4	Executar endpoint Clicar em qualquer rota → 'Try it out' → preencher → Execute	Resposta com status HTTP e JSON
5	Trocar usuário Clicar 'Authorize' → Logout → login com outro usuário	Sessão trocada; útil para testar BOLA

Campos opcionais no Swagger

- Campos marcados como 'nullable' podem ser omitidos — basta deixar em branco ou null.
- llm_provider: omitir usa o LLM_DEFAULT_PROVIDER do .env.
- target_dim no benchmark: omitir usa original_dim // 2.

3. Modo Mock — CI/Dev Offline

O Mock é o provider padrão em ambiente de desenvolvimento (**LLM_DEFAULT_PROVIDER=mock**). Não requer Ollama, OpenAI ou qualquer rede externa. Ideal para validar o pipeline completo de forma determinística e sem custo. A resposta é sempre a mesma para o mesmo input.

Configuração necessária

- LLM_DEFAULT_PROVIDER=mock no .env (já é o default)
- Nenhuma variável adicional necessária
- just dev para subir o servidor

3.1 Fluxo completo com Mock

1	Registro POST /api/v1/auth/register {email: 'adv@exemplo.com', password: 'SenhaForte@2026!'}	201 — mesmo código para email existente (anti-enumeração)
2	Login POST /api/v1/auth/login {email: 'adv@exemplo.com', password: 'SenhaForte@2026!'}	200 com access_token JWT
3	Providers de embedding GET /api/v1/documents/providers	Lista com sentence_transformers (e openai se OPENAI_API_KEY setada)
4	Upload de TXT jurídico POST /api/v1/documents file: contrato.txt, embedding_provider: sentence_transformers	201 com status=embedded e chunks_created >= 1
5	Listar documentos GET /api/v1/documents	Array com o documento recém-criado
6	Providers de LLM GET /api/v1/queries/llm-providers	Items: [mock, ollama]. Default: mock
7	Ask com Mock POST /api/v1/queries/ask {"query": "Qual o prazo de rescisão?", "embedding_provider": "sentence_transformers", "llm_provider": "mock"}	200 com answer=[MOCK]..., model=mock, cost_usd=0.0, tokens_used=0, grounded=true
8	Benchmark compressão POST /api/v1/documents/{id}/benchmark-compression {}	200 com 4 resultados, savings_pct > 0, best_method válido
9	Dashboard GET /api/v1/metrics/dashboard	llm_stats com provider=mock, document_stats.embedded=1
10	Benchmark embedding POST /api/v1/metrics/benchmark-embedders {"query": "rescisão contratual"}	200 com sentence_transformers, latency_ms real, cost=0.0

3.2 Resultado esperado do ask com Mock

```
{
  "query": "Qual o prazo de rescisão?",
  "answer": "[MOCK] Resposta determinística com base no trecho [1]...",
  "model": "mock",
  "llm_provider": "mock",
```

```
"tokens_used": 0,  
"cost_usd": 0.0,  
"latency_ms": 1,  
"grounded": true,  
"sources": [  
  {  
    "citation_number": 1,  
    "document_filename": "contrato.txt",  
    "similarity": 0.85  
  }  
]  
}
```

4. Modo Ollama — LLM Local (Sem Custo, Sem Rede)

O Ollama executa o modelo **qwen2.5:7b** inteiramente na máquina local. Nenhum dado sai para servidores externos. Custo zero. Ideal para demonstrar o argumento central do projeto: provider pluggable com execução offline.

Configuração necessária

- Ollama instalado: `curl -fsSL https://ollama.com/install.sh | sh`
- Modelo baixado: `ollama pull qwen2.5:7b` (ou `llama3.2:3b` para máquina fraca)
- Servidor rodando: `ollama serve` (Terminal separado do just dev)
- No `.env`: `LLM_DEFAULT_PROVIDER=ollama` (ou passar `llm_provider=ollama` no body)

4.1 Verificar se o Ollama está respondendo antes do teste

```
# Em outro terminal, antes de testar:
curl http://localhost:11434/api/tags
# Esperado: JSON com lista de modelos instalados
# Se retornar erro de conexão: ollama serve não está rodando
```

4.2 Fluxo de Ask com Ollama

1	Login + Authorize Mesmo fluxo da seção 3	Token JWT válido
2	Upload documento Mesmo fluxo da seção 3 (se já fez, pular)	<code>status=embedded</code>
3	Providers de LLM <code>GET /api/v1/queries/llm-providers</code>	Items inclui 'ollama' com <code>model=qwen2.5:7b</code>
4	Ask com Ollama <code>POST /api/v1/queries/ask {"query": "Qual o prazo de notificação para rescisão?", "embedding_provider": "sentence_transformers", "llm_provider": "ollama"}</code>	200 com resposta em português, <code>grounded=true, cost_usd=0.0, tokens_used > 0</code>
5	Verificar latência Campo <code>latency_ms</code> no response	Entre 1500ms e 8000ms dependendo do hardware

4.3 Resultado esperado do ask com Ollama

```
{
  "query": "Qual o prazo de notificação para rescisão?",
  "answer": "Conforme a Cláusula Terceira do contrato [1], o prazo de notificação\nprévia para rescisão é de 30 (trinta) dias...",
  "model": "qwen2.5:7b",
  "llm_provider": "ollama",
  "tokens_used": 127,
  "cost_usd": 0.0,
  "latency_ms": 3200,
  "grounded": true,
  "sources": [{"citation_number": 1, "document_filename": "contrato.txt"}]
}
```

4.4 Comportamento esperado se Ollama não estiver rodando

Erro esperado (502 Bad Gateway)

- Se ollama serve não estiver ativo ao chamar `llm_provider=ollama`:
- HTTP 502 com mensagem: 'Falha na chamada ao Ollama em http://localhost:11434'
- Isso valida o handler de erro do sistema — comportamento correto.
- Solução: abrir terminal, rodar 'ollama serve', tentar novamente.

5. Modo OpenAI — Cloud / Alta Qualidade

O modo OpenAI usa **gpt-4o-mini** via API da OpenAI. Requer chave de API e conexão com a internet. Tem custo por token mas oferece a maior qualidade de resposta. Ideal para demonstrar o modo 'ultra' da plataforma e comparar com Ollama no dashboard.

Configuração necessária

- OPENAI_API_KEY=sk-... no .env
- LLM_DEFAULT_PROVIDER=openai (ou passar llm_provider=openai no body)
- Sem OPENAI_API_KEY: o provider não aparece em /llm-providers e retorna 502 se forçado

5.1 Fluxo de Ask com OpenAI

1	Verificar key configurada GET /api/v1/queries/llm-providers	Items deve incluir 'openai' com model=gpt-4o-mini
2	Ask com OpenAI POST /api/v1/queries/ask {"query": "Quais são as obrigações das partes?", "embedding_provider": "sentence_transformers", "llm_provider": "openai"}	200 com resposta jurídica detalhada, tokens_used > 0, cost_usd > 0
3	Verificar custo Campo cost_usd no response	Valor > 0 (ex.: 0.000045 para query simples)

5.2 Resultado esperado do ask com OpenAI

```
{
  "query": "Quais são as obrigações das partes?",
  "answer": "As partes têm as seguintes obrigações conforme os termos contratuais [1][2]:\n  A CONTRATANTE obriga-se a efetuar o pagamento mensal [1]...",
  "model": "gpt-4o-mini",
  "llm_provider": "openai",
  "tokens_used": 284,
  "cost_usd": 0.0000426,
  "latency_ms": 820,
  "grounded": true
}
```

5.3 Comportamento sem API Key

Erro esperado (502 Bad Gateway)

- Se OPENAI_API_KEY não estiver setada e llm_provider=openai for enviado:
- HTTP 502 com mensagem: 'Provedor OpenAI selecionado, mas OPENAI_API_KEY não está configurada'
- provider 'openai' não aparece em GET /llm-providers
- Comportamento correto — não vazar informação sobre configuração ausente como 500.

6. Sprint 4A — Benchmark de Compressão de Embeddings

O endpoint de benchmark executa os 4 compressores (PCA, RandomProjection, Int8, Binary) sobre os embeddings reais do documento, calcula a retenção semântica via Pearson de matrizes de similaridade coseno e persiste o melhor resultado.

6.1 Executar o benchmark

1	Endpoint POST /api/v1/documents/{id}/benchmark-compression Body: {} (target_dim opcional)	200 OK com 4 resultados
2	Com target_dim customizado Body: {"target_dim": 64}	PCA e RP com compressed_dim=64; Int8 e Binary preservam dimensão
3	Verificar savings Campo savings_pct no response	Valor > 0 — confirma economia de armazenamento
4	Verificar banco SELECT compression_method, original_dimension, compressed_dimension, semantic_retention FROM documents WHERE id='{id}'	Colunas preenchidas pelo benchmark

6.2 Resultado esperado do benchmark

```
{
  "document_id": "...",
  "best_method": "int8",
  "best_semantic_retention": 0.9834,
  "original_storage_mb": 0.0015,
  "best_storage_mb": 0.000375,
  "savings_pct": 75.0,
  "results": [
    {
      "method": "int8",
      "semantic_retention": 0.9834,
      "space_savings_pct": 75.0,
    },
    {
      "method": "pca",
      "semantic_retention": 0.9721,
      "space_savings_pct": 50.0,
    },
    {
      "method": "random_projection",
      "semantic_retention": 0.9480,
      "space_savings_pct": 50.0,
    },
    {
      "method": "binary",
      "semantic_retention": 0.7210,
      "space_savings_pct": 96.875
    }
  ]
}
```

6.3 Interpretação para a banca

Método	Dimensão	Bytes/elem	Ratio	Economia	Retenção	Quando usar
Int8	384→384	1 byte	4×	75.0%	0.983	Melhor custo-benefício: custo zero de qualidade vs 4x de economia
PCA	384→192	4 bytes	2×	50.0%	0.972	Reduz dimensão à metade mantendo alta qualidade semântica

RandomProjection	384→192	4 bytes	2×	50.0%	0.948	Mais rápido que PCA no fit, levemente inferior em retenção
Binary	384→384	0.125 B	32×	96.9%	0.721	Máxima economia teórica (packbits), maior perda semântica

7. Sprint 4B — Dashboard de KPIs e Benchmark de Embedding

7.1 Dashboard de métricas

Agrega **audit_logs.metadata** em KPIs: latência por provider LLM, custo acumulado, retenção semântica por compressor e economia de armazenamento em Pinecone/Qdrant.

1	Endpoint GET /api/v1/metrics/dashboard	200 OK com todos os KPIs
2	Sem dados ainda GET logo após seed, sem nenhuma query executada	total_queries=0, storage_savings=null — comportamento correto
3	Após ask e benchmark GET após executar /ask e /benchmark-compression	llm_stats preenchido, storage_savings preenchido

```
{
  "total_queries": 3,
  "total_cost_usd": 0.0,
  "total_cost_brl": 0.0,
  "llm_stats": [
    {"provider": "ollama", "total_calls": 2, "avg_latency_ms": 3400.0,
      "total_tokens": 284, "total_cost_usd": 0.0},
    {"provider": "mock", "total_calls": 1, "avg_latency_ms": 1.0,
      "total_tokens": 0, "total_cost_usd": 0.0}
  ],
  "compression_stats": [
    {"method": "int8", "avg_semantic_retention": 0.983, "avg_space_savings_pct": 75.0}
  ],
  "document_stats": {"total": 1, "embedded": 1, "failed": 0},
  "storage_savings": {"savings_pct": 75.0,
    "pinecone_original_usd": 0.0000033, "qdrant_savings_brl": 0.0}
}
```

7.2 Benchmark de provedores de embedding

1	Endpoint POST /api/v1/metrics/benchmark-embedders {"query": "rescisão contratual por justa causa"}	200 OK com latência real de cada provider
2	Resultado esperado sentence_transformers: latency_ms ~50-200ms, cost=0.0	cheapest_provider: sentence_transformers
3	Com OpenAI disponível Mesma query; openai aparece nos results	openai: latency ~100-400ms, cost_usd_per_million_tokens > 0

```
{
  "query": "rescisão contratual por justa causa",
  "fastest_provider": "sentence_transformers",
  "cheapest_provider": "sentence_transformers",
  "results": [
    {
      "provider_name": "sentence_transformers",
      "model_name": "intfloat/multilingual-e5-small",
      "dimensions": 384,
      "latency_ms": 87,
      "cost_usd_per_million_tokens": 0.0
    }
  ]
}
```

```
]
}
```

8. Validação Direta no Banco de Dados

Após executar o fluxo completo, confirmar no banco que os dados foram persistidos corretamente. Executar: **just psql**

8.1 Documento com compressão preenchida

```
SELECT filename, status, compression_method,
       original_dimension, compressed_dimension,
       ROUND(CAST(semantic_retention AS numeric), 4) AS retention
FROM documents
WHERE owner_id = (SELECT id FROM users WHERE role = 'user' LIMIT 1);
```

Campo	Valor esperado
filename	contrato.txt
status	embedded
compression_method	int8 (ou o melhor resultado do benchmark)
original_dimension	384
compressed_dimension	384 (Int8 preserva dimensão)
retention	~0.983 (Pearson de cosine similarities)

8.2 Audit log — rastreabilidade completa

```
SELECT action, status,
       event_metadata->>'llm_provider' AS llm_provider,
       event_metadata->>'benchmark_type' AS bench_type,
       event_metadata->>'best_method' AS best_method,
       event_metadata->>'latency_ms' AS latency_ms,
       event_metadata->>'cost_usd' AS cost_usd
FROM audit_logs
ORDER BY created_at DESC
LIMIT 10;
```

action	status	llm_provider	bench_type	best_method	latency_ms	cost_usd
query_executed	success	ollama	—	—	3400	0.0
query_executed	success	mock	—	—	1	0.0
document_updated	success	—	compression	int8	—	—
document_updated	success	—	embedding	—	—	—
document_uploaded	success	—	—	—	—	—
user_registered	success	—	—	—	—	—

9. Testes de Segurança Manuais

Os testes automatizados cobrem os cenários de segurança, mas esta seção documenta como reproduzi-los manualmente no Swagger para demonstração na banca.

9.1 Anti-enumeração no /register

1	Email novo POST /register {email: "novo@exemplo.com", password: "Forte@2026!"}	201 — mensagem genérica
2	Email existente POST /register {email: "novo@exemplo.com", password: "OutraForte@2026!"}	201 — MESMA mensagem genérica, mesmo status
3	Senha fraca POST /register {email: "outro@exemplo.com", password: "123"}	400/422 — rejeitado por política NIST SP 800-63B

9.2 BOLA — Isolamento multi-tenant

1	Login usuário A POST /auth/login com usuário A	Token de A
2	Upload com A POST /documents com token de A	Document ID pertencente a A
3	Login usuário B POST /auth/login com usuário B (Authorize com token de B)	Token de B
4	Tentar acessar doc de A GET /documents/{id_de_A} com token de B	404 — mascaramento RGN-10 (não expõe 403)
5	Benchmark com token errado POST /documents/{id_de_A}/benchmark-compression com token de B	404 — mesma defesa BOLA

9.3 Rate limit

1	Disparar 12 asks POST /ask 12 vezes em sequência rápida (cURL loop ou repetir no Swagger)	Primeiras 10 retornam 200; a partir da 11ª retorna 429
2	Sem token GET /documents sem Authorization header	401 Unauthorized

10. Checklist de Aprovação para Sprint 5

Marcar todos os itens abaixo antes de iniciar o frontend. O sistema está pronto quando todos estiverem confirmados.

	Item de validação	Prioridade
■	just ci verde	Obrigatório
■	Banco rodando, migrations até 0006	Obrigatório
■	Upload de documento com status = embedded	Obrigatório
■	GET /queries/llm-providers retorna mock e ollama	Obrigatório
■	POST /ask com llm_provider=mock retorna [MOCK]...	Obrigatório
■	POST /ask com llm_provider=ollama retorna PT-BR	Recomendado (requer Ollama)
■	POST /benchmark-compression retorna 4 compressores	Obrigatório
■	GET /metrics/dashboard retorna llm_stats preenchido	Obrigatório
■	POST /benchmark-embedders retorna latência real	Obrigatório
■	documents.compression_method preenchido no banco	Obrigatório
■	audit_logs com query_executed e benchmark	Obrigatório
■	Anti-enumeração: 201 para email duplicado	Obrigatório
■	BOLA: 404 ao acessar doc de outro usuário	Obrigatório

Pronto para Sprint 5 quando

- Todos os itens 'Obrigatório' estiverem com ■.
- O audit_log tiver pelo menos 1 query_executed e 1 benchmark compression.
- O dashboard retornar storage_savings com savings_pct > 0.
- A resposta do Ollama (ou Mock) vier em português com citação [1].